Machine Learning Techniques

Unit 5

Ankita Singh, Dept. of IT, AKGEC

Syllabus

REINFORCEMENT LEARNING–Introduction to Reinforcement Learning , Learning Task, Example of Reinforcement Learning in Practice, Learning Models for Reinforcement –(Markov Decision process , Q Learning - Q Learning function, Q Learning Algorithm), Application of Reinforcement Learning,Introduction to Deep Q Learning.

GENETIC ALGORITHMS: Introduction, Components, GA cycle of reproduction, Crossover,Mutation, Genetic Programming, Models of Evolution and Learning, Applications.

Reinforcement learning

- *Reinforcement learning is a feedback-based learning method, in which a learning agent gets a reward for each right action and gets a penalty for each wrong action.*
- The agent learns automatically with these feedbacks and improves its performance. In reinforcement learning, the agent interacts with the environment and explores it.
- The goal of an agent is to get the ***most*** **reward points**, and hence, it improves its performance.
- The robotic dog, which automatically learns the movement of his arms, is an example of Reinforcement learning.

- As compared to unsupervised learning, reinforcement learning is different in terms of goals. While the goal in **unsupervised learning** is to find similarities and differences between data points. RL is a technique that enables an agent to learn in an interactive **environment** by trial and error feedback from its own actions and experiences.
- Environment can be real world (robot) or simulated world (ex. Video game).
- In reinforcement learning the goal is to find a suitable action model that would **maximize the total cumulative reward** of the agent.
- The agent learns without intervention from a human by maximizing its reward and minimizing its penalty.
- RL can guide an agent on how to act in the real world (robot that vacuum your house, robot that learn to walk).
- Military is interested in this tech: RL agent can replace soldiers not just walk, but fight, diffuse bombs, make important decisions.

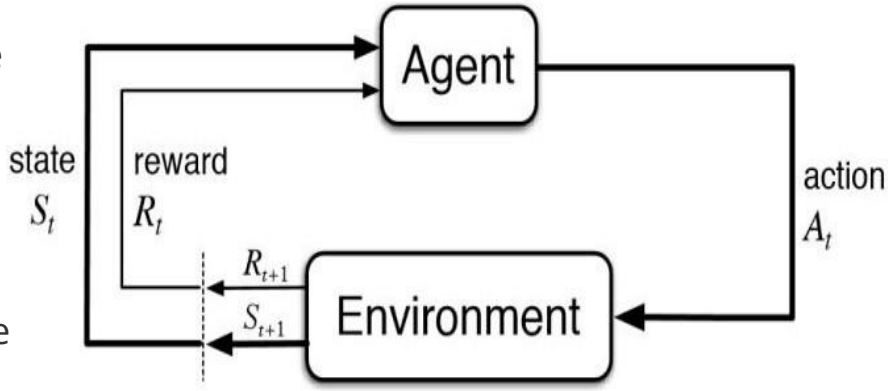
Reinforcement learning

Reinforcement Learning system is comprised of two main components:

- Agent
- Environment

It is Inspired by human brain. We have eyes and nose as sensory organ but machine has sensors as sensory organ.

- **Agent-** an important component to process the information, thing that senses the environment.
- **Environment** – real world or simulated world that the agent lives in.
- **State**- different configuration of the environment that the agent can sense.
- **Actions**: are what an agent does in its environment. Notation (s,a,r).
- **Rewards**- positive or negative.



Reinforcement Learning Definitions

Agent: The RL algorithm that learns from trial and error

Environment: The world through which the agent moves

Action (A): All the possible steps that the agent can take

State (S): Current condition returned by the environment

Reinforcement Learning Definitions

Reward (R): An instant return from the environment to appraise the last action

Policy (π): The approach that the agent uses to determine the next action based on the current state

Value (V): The expected long-term return with discount, as opposed to the short-term reward R

Action-value (Q): This similar to Value, except, it takes an extra parameter, the current action (A)

Types of Reinforcement learning

Reinforcement learning is categorized mainly into two types of methods/algorithms:

- **Positive Reinforcement Learning**: Positive reinforcement learning specifies increasing the tendency that the **required behavior would occur again by adding something.** It enhances the strength of the behavior of the agent and positively impacts it.

- **Negative Reinforcement Learning:** Negative reinforcement learning works exactly opposite to the positive RL. It increases the tendency that the specific behavior would occur again by **avoiding the negative condition.**

Learning Task in Reinforcement Learning

·Consider building a **learning robot**. The robot, or ***agent***, has a set of *sensors* to observe the state of its environment, and a *set of actions* it can perform to alter this state.

·Its task is to learn a control strategy, or ***policy***, for choosing actions that achieve its goals.

·The goals of the agent can be defined by a ***reward function*** that assigns a numerical value to each distinct action the agent may take from each distinct state.

·This reward function may be built into the robot, or known only to an external teacher who provides the reward value for each action performed by the robot.

·The **task** of the robot is to perform sequences of actions, observe their consequences, and learn a control policy.

·The control policy is one that, from any initial state, chooses actions that maximize the reward accumulated over time by the agent.

Learning Task in Reinforcement Learning

The learning task in RL involves an agent trying to find an optimal sequence of actions within an environment to achieve a specific goal. This typically involves:

- An agent interacting with its environment. The agent exists in an environment described by some set of possible states S.
- Agent perform any of a set of possible actions A. Each time it performs an action a, in some state st the agent receives a real-valued reward r, that indicates the immediate value of this state-action transition. This produces a sequence of states si, actions ai, and immediate rewards ri as shown in the figure.
- The agent's task is to learn a control policy, π**: S** $\rightarrow$ **A**, that maximizes the expected sum of these rewards, with future rewards discounted exponentially by their delay.

Reinforcement Learning Concepts

1. **Reward Maximization**

   This theory states that, a RL agent must be trained in such a way that it takes the best action so that the reward is maximum.

2. **Exploration and Exploitation**

   Exploitation is about using already known exploited information to heighten the reward. Exploration is about exploring and capturing more information about the environment.

Reinforcement Learning Examples

1. **Automated Robots**

   Robots trained in RL can learn to navigate an environment, grasp objects, and move them. Automated robots can perform tasks that are too dangerous or impossible for humans. They can also perform some tasks faster and more efficiently than humans, such as scanning warehouse inventory and other aspects of supply chain optimization.

2. **Financial Trading**

   Financial analysts can use reinforcement learning to drive profitable trading strategies. The most innovative methods use multi-agent systems to improve strategies by simulating multiple scenarios in a market environment

3. Marketing and Advertising

Marketers test multiple versions of a campaign to determine which is most effective. Reinforcement learning automates this process, and agents can automatically allocate more traffic to higher-performing versions in real time.

4. Gaming

Gaming has been one of the primary use cases for RL in the past. Many of the developments that have come from the gaming industry, such as natural language processing and intelligent agents, are now being applied to wider use cases.

Markov Decision process

A **Markov Decision Process (MDP)** is a mathematical framework used in Reinforcement Learning (RL) to model decision-making problems. It assumes the **Markov Property**, which states:

> *the next state depends only on the current state and action, not on past states.*

Mathematically,

$$P(s' \mid s, a, s_{t-1}, a_{t-1}, \ldots) = P(s' \mid s, a)$$

Thus, MDP provides a formal structure for an agent to interact with an environment and learn an optimal strategy (policy) to maximize cumulative reward.

Components of an MDP

An MDP is defined by a tuple (S, A, P, R, γ) where:

- **S (State Space):** A finite or infinite set of states representing the environment.
- **A (Action Space):** A set of actions available to the agent in each state.
- **P (Transition Probability):** A probability function **P(s' | s, a)** that defines the likelihood of transitioning from state **s** to **s'** after taking action **a**.
- **R (Reward Function):** A function **R(s, a, s')** that assigns a reward for moving from state **s** to **s'** via action **a**.
- **γ (Discount Factor):** A value in the range **[0,1]** that determines the importance of future rewards.

Objective of Agent in MDP is to maximize the expected cumulative reward.

Policy (π)**:** A policy defines the agent's strategy for selecting best actions in each state.

Value Functions: Used to evaluate the quality of states or state-action pairs:

1. State Value Function (Vπ(s)): It represents the expected cumulative reward from state S under policy π.

2. Action Value Function (*Qπ*(*s*,*a*)**):** It evaluates the expected reward of taking action **a** in state **s** and following π thereafter.

MDP provides the core mathematical foundation for reinforcement learning.
It models the environment, rewards, and transitions while ensuring decisions depend only on the present state.

Q Learning

Q-Learning is a popular model-free reinforcement learning algorithm that helps an agent learn how to make the best decisions by interacting with its environment.

Instead of needing a model of the environment the agent learns purely from experience by trying different actions and seeing their results.

Q-learning is a model-free, value-based, off-policy algorithm that will find the best series of actions based on the agent's current state. The “Q” stands for quality. Quality represents how valuable the action is in maximizing future rewards.

Terminologies in Q-learning

- **States(s)**: the current position of the agent in the environment.
- **Action(a)**: a step taken by the agent in a particular state.
- **Rewards**: for every action, the agent receives a reward and penalty.
- **Episodes**: the end of the stage, where agents can't take new action. It happens when the agent has achieved the goal or failed.
- **Q-Table**: the agent maintains the Q-table of sets of states and actions.
- **Temporal Differences(TD)**: used to estimate the expected value of $Q(S_{t+1}, a)$ by using the current state and action and previous state and action.

Q learning algorithm

For each s, a initialize the table entry $\hat{Q}(s, a)$ to zero.
Observe the current state s
Do forever:

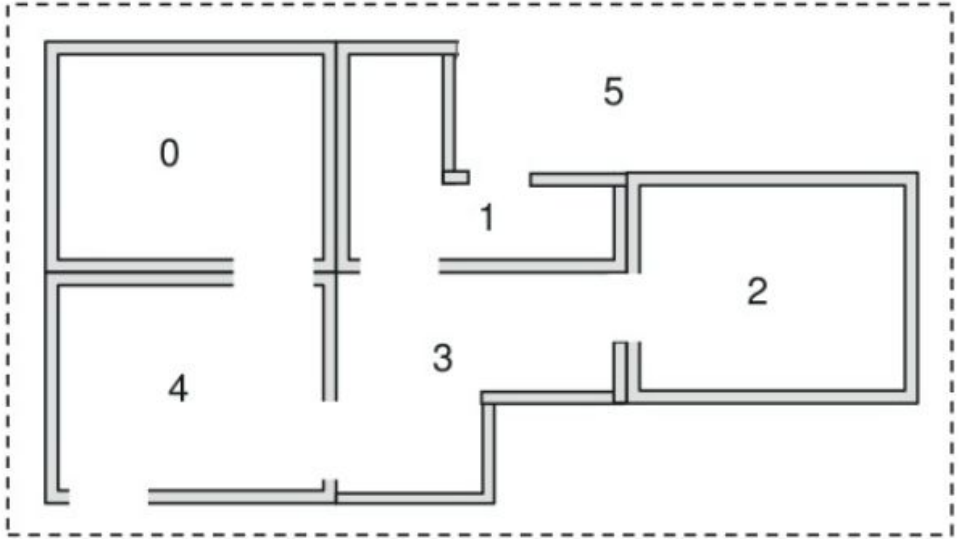
- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$

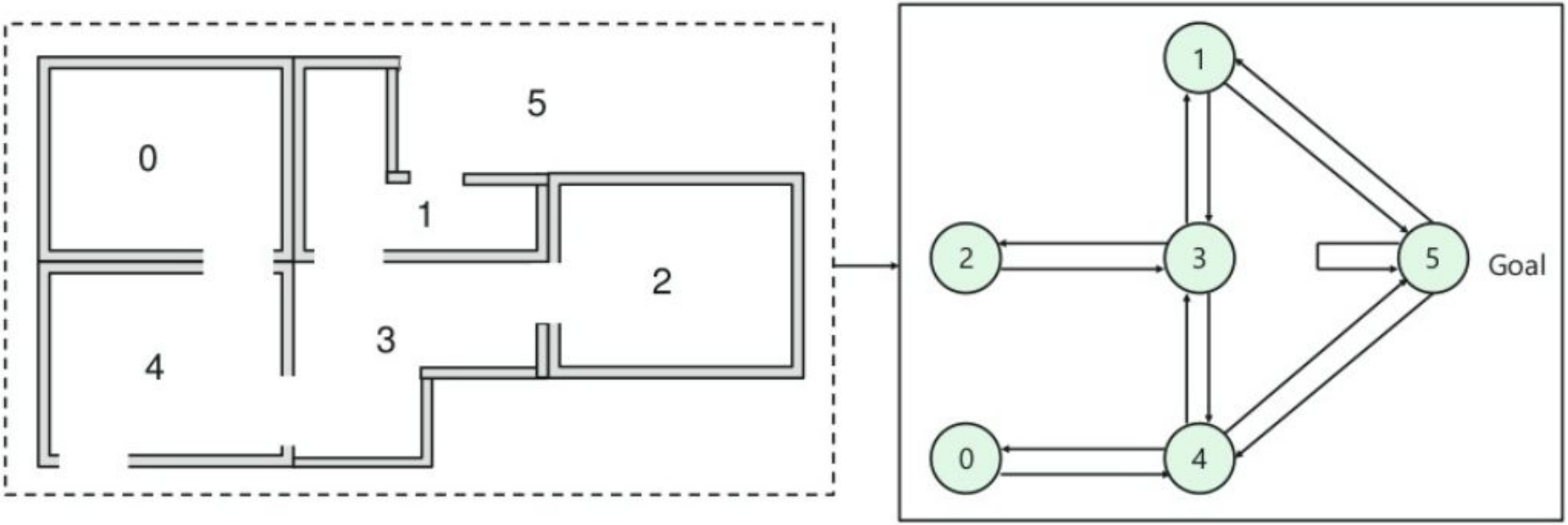
Example

Place an agent in any one of the rooms (0,1,2,3,4) and the goal is to reach outside the building (room 5)



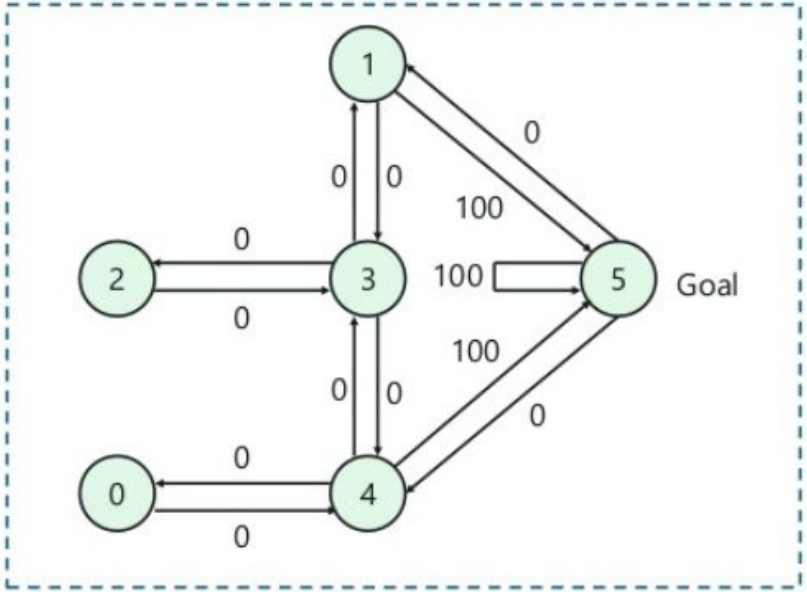
- 5 rooms in a building connected by doors
- each room is numbered 0 through 4
- The outside of the building can be thought of as one big room (5)
- Doors 1 and 4 lead into the building from room 5 (outside)

Let's represent the rooms on a graph, each room as a node, and each door as a link



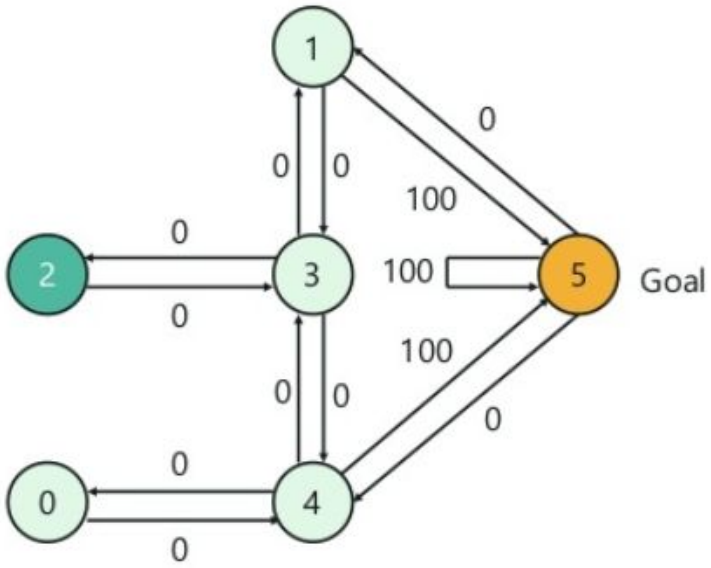
Next step is to associate a reward value to each door:

- doors that lead directly to the goal have a reward of 100
- Doors not directly connected to the target room have zero reward
- Because doors are two-way, two arrows are assigned to each room
- Each arrow contains an instant reward value



The terminology in Q-Learning includes the terms state and action:

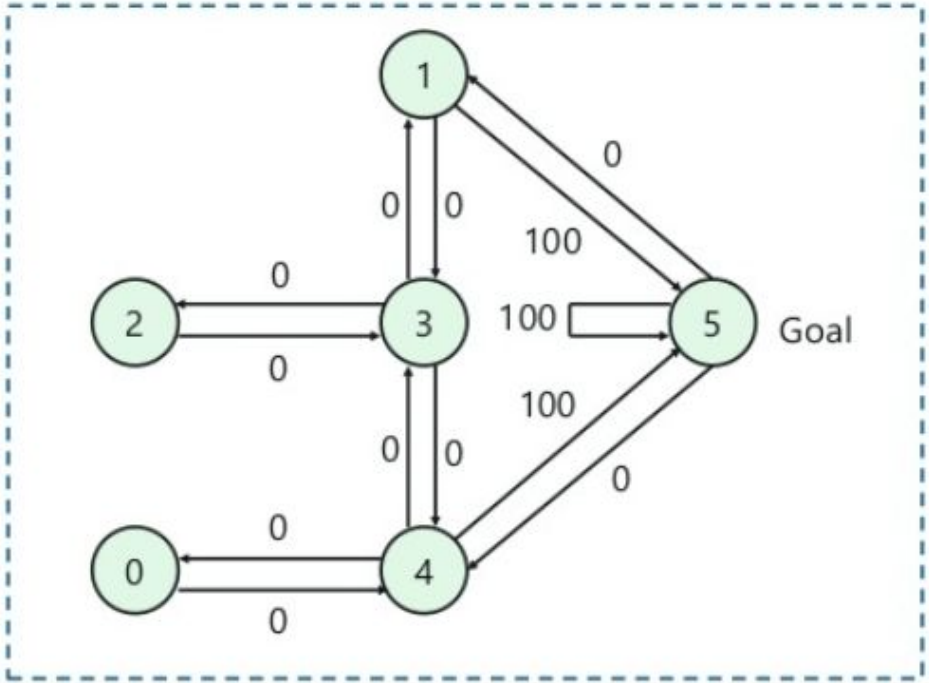
- Room (including room 5) represents a state
- agent's movement from one room to another represents an action
- In the figure, a state is depicted as a node, while "action" is represented by the arrows



Example (Agent traverse from room 2 to room5):

1. Initial state = state 2
2. State 2 -> state 3
3. State 3 -> state (2, 1, 4)
4. State 4 -> state 5

We can put the state diagram and the instant reward values into a reward table, matrix R.



$$R = \begin{array}{c} \\ \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Action} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array}$$

The -1's in the table represent null values

Add another matrix Q, representing the memory of what the agent has learned through experience.

- The rows of matrix Q represent the current state of the agent
- columns represent the possible actions leading to the next state
- Formula to calculate the Q matrix:

$$Q(state, action) = R(state, action) + Gamma * Max\ [Q(next\ state, all\ actions)]$$

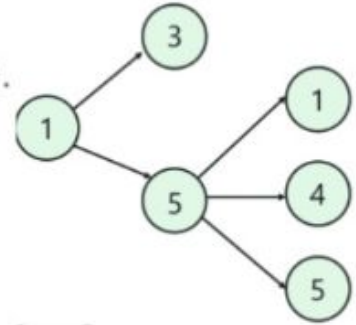
Note

The Gamma parameter has a range of 0 to 1 (0 <= Gamma > 1).

- If Gamma is closer to zero, the agent will tend to consider only immediate rewards.
- If Gamma is closer to one, the agent will consider future rewards with greater weight

1. Set the gamma parameter, and environment rewards in matrix R
2. Initialize matrix Q to zero
3. Select a random initial state
4. Set initial state = current state
5. Select one among all possible actions for the current state
6. Using this possible action, consider going to the next state
7. Get maximum Q value for this next state based on all possible actions
8. Compute: Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]
9. Repeat above steps until current state = goal state

First step is to set the value of the learning parameter Gamma = 0.8, and the initial state as Room 1.



Next, initialize matrix Q as a zero matrix:

- From room 1 you can either go to room 3 or 5, let's select room 5.
- From room 5, calculate maximum Q value for this next state based on all possible actions:

*Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]*

Q(1,5) = R(1,5) + 0.8 * Max[Q(5,1), Q(5,4), Q(5,5)] = 100 + 0.8 * 0 = 100

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

$$R = \begin{array}{c|cccccc} \text{State} \backslash \text{Action} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 100 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

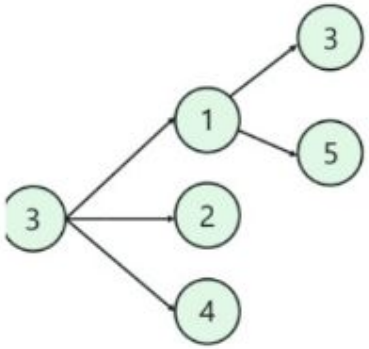
For the next episode, we start with a randomly chosen initial state, i.e. state 3

- From room 3 you can either go to room 1,2 or 4, let's select room 1.
- From room 1, calculate maximum Q value for this next state based on all possible actions:

*Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]*

Q(3,1) = R(3,1) + 0.8 * Max[Q(1,3), Q(1,5)]= 0 + 0.8 * [0, 100] = 80

The matrix Q get's updated



$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 100 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

$$R = \begin{array}{c|cccccc} \text{State} \backslash \text{Action} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 100 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 80 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

the next state, 1, now becomes the current state. We repeat the inner loop of the Q learning algorithm because state 1 is not the goal state.

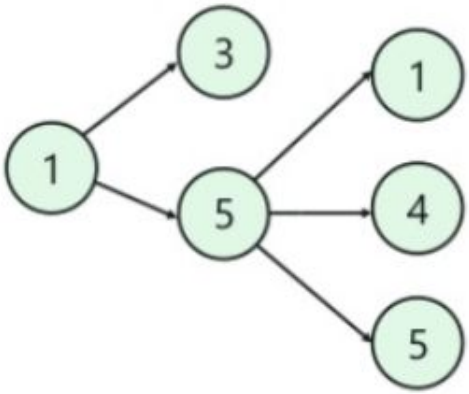
- From room 1 you can either go to room 3 or 5, let's select room 5.
- From room 5, calculate maximum Q value for this next state based on all possible actions:

*Q(state, action) = R(state, action) + Gamma * Max[Q(next state, all actions)]*

Q(1,5) = R(1,5) + 0.8 * Max[Q(5,1), Q(5,4), Q(5,5)] = 100 + 0.8 * 0 = 100
The matrix Q remains the same since, Q(1,5) is already fed to the agent

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$R = \begin{array}{c} \text{State} \\ \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \text{Action} \\ \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{array}$$

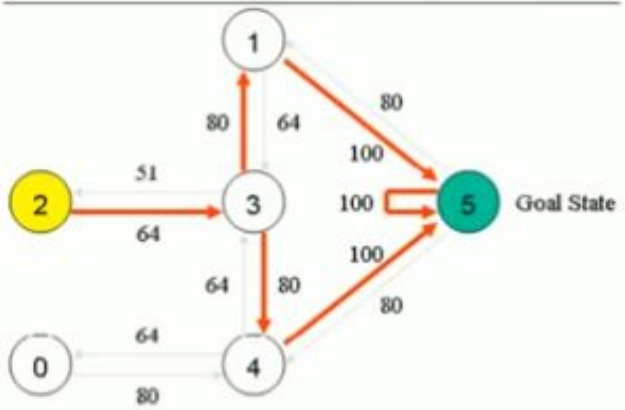


No Change in Q

- If our agent learns more through further episodes, it will finally reach convergence values in matrix Q like:

$$Q = \begin{array}{c} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{array} \begin{array}{c} \begin{array}{cccccc} 0 & 1 & 2 & 3 & 4 & 5 \end{array} \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{array}$$

- Tracing the best sequences of states is as simple as following the links with the highest values at each state.



Application of Reinforcement Learning

- **Robotics**: RL is used to automate tasks in structured environments such as manufacturing, where robots learn to optimize movements and improve efficiency.
- **Games**: Advanced RL algorithms have been used to develop strategies for complex games like chess, Go and video games, outperforming human players in many instances.
- **Industrial Control**: RL helps in real-time adjustments and optimization of industrial operations, such as refining processes in the oil and gas industry.
- **Personalized Training Systems**: RL enables the customization of instructional content based on an individual's learning patterns, improving engagement and effectiveness.

Introduction to Deep Q Learning

- **One of the main drawbacks of Q-learning is that it becomes infeasible when dealing with large state spaces, as the size of the Q-table grows exponentially with the number of states and actions**.
- In such cases, the algorithm becomes computationally expensive and requires a lot of memory to store the Q-values.
- Imagine a game with 1000 states and 1000 actions per state. We would need a table of one million cells. And that is a very small state space compared to chess or Go.
- Also, Q-learning can’t be used in unknown states because it can’t infer the Q-value of new states from the previous ones.

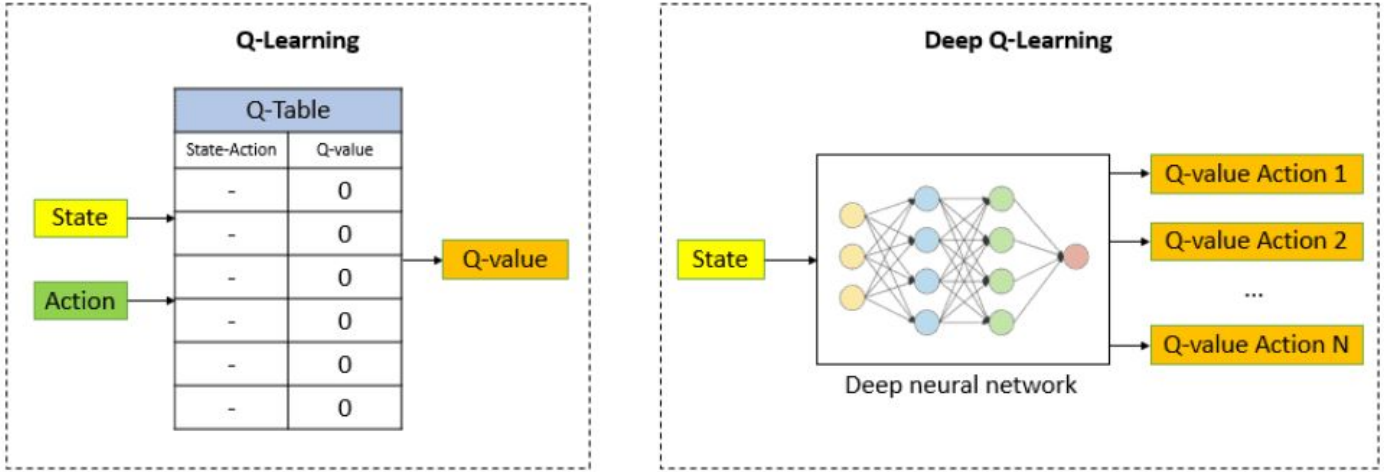
This presents two problems:

First, the amount of memory required to save and update that table would increase as the number of states increases.

Second, the amount of time required to explore each state to create the required Q-table would be unrealistic.

- To tackle this challenge, one alternative approach is to combine Q-learning with deep neural networks. This approach is coined as Deep Q-Learning (DQL). The neural networks in DQL act as the Q-value approximator for each (state, action) pair.
- Deep Q-Learning is a method that uses deep learning to help machines make decisions in complicated situations. It’s especially useful in environments where the number of possible situations called states is very large like in video games or robotics.

The neural network receives the state as an input and outputs the Q-values for all possible actions. The following figure illustrates the difference between Q-learning and deep Q-learning in evaluating the Q-value:



Essentially, deep Q-Learning replaces the regular Q-table with the neural network. Rather than mapping a (state, action) pair to a Q-value, the neural network maps input states to (action, Q-value) pairs.

In 2013, DeepMind introduced Deep Q-Network (DQN) algorithm. DQN is designed to learn to play Atari games from raw pixels. This was a breakthrough in the field of reinforcement learning and helped pave the way for future developments in the field. **The term Deep Q-network refers to the neural network in their DQL architecture**.

Challenges Addressed by Deep Q-Learning

- **High-Dimensional State Spaces:** Traditional Q-Learning uses a table to store values but this becomes impossible when there are too many situations. Neural networks can understand and work with many different situations at once so they are better for complex problems.
- **Continuous Input Data:** Real-world problems often have continuous data like video images. Neural networks are good at handling this kind of information.
- **Scalability:** Deep learning helps Q-Learning grow and handle bigger, harder tasks that regular Q-Learning couldn’t solve before.

How Deep Q-Learning Work

Here are the steps of how DQN works:

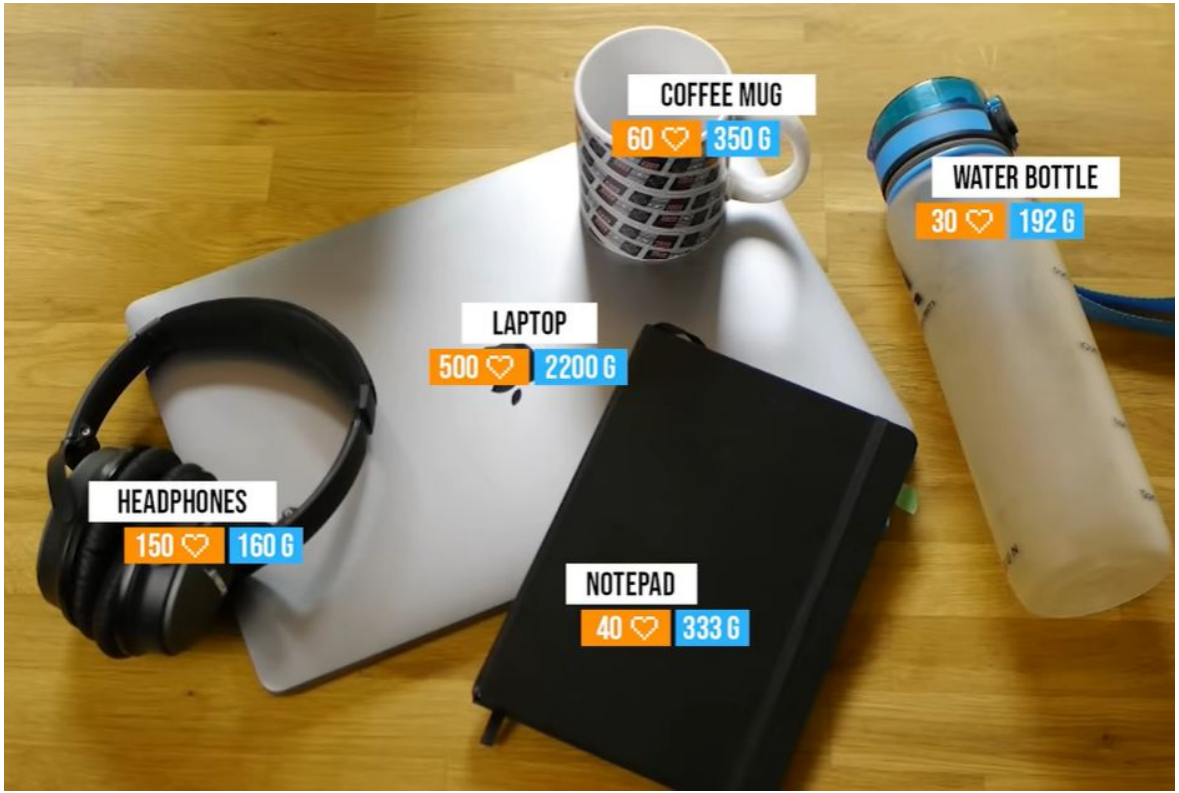
- **Environment**: DQN interacts with an environment with a state, an action space, and a reward function. The goal of the DQN is to learn the optimal policy that maximizes cumulative rewards over time
- **Replay Memory**: DQN uses a replay memory buffer to store past experiences. Each experience is a tuple (state, action, reward, next state) representing a single transition from one state to another. The replay memory stores these experiences to sample from later randomly
- **Deep Neural Network**: DQN uses a deep neural network to estimate the Q-values for each (state, action) pair. The neural network takes the state as input and outputs the Q-value for each action. The network is trained to minimize the difference between the predicted and target Q-values

- **Epsilon-Greedy Exploration**: DQN uses an **epsilon-greedy exploration strategy** to balance exploration and exploitation. During training, the agent selects a random action with probability epsilon and selects the action with the highest Q-value with probability (1 – epsilon)
- **Target Network**: DQN uses a separate target network to estimate the target Q-values. The target network is a copy of the main neural network with fixed parameters. The target network is updated periodically to prevent the overestimation of Q-values
- **Training**: DQN trains the neural network using the Bellman equation to estimate the optimal Q-values. The loss function is the mean squared error between the predicted and target values. The target Q-value is calculated using the target network and the Bellman equation. The neural network weights are updated using backpropagation and stochastic gradient descent
- **Testing**: DQN uses the learned policy to make environmental decisions after training. The agent selects the action with the highest Q-value for a given state

In summary, DQN learns the optimal policy by using a deep neural network to estimate the Q-values, a replay memory buffer to store past experiences, and a target network to prevent overestimating Q-values. The agent uses an epsilon-greedy exploration strategy during training and selects the action with the highest Q-value during testing.

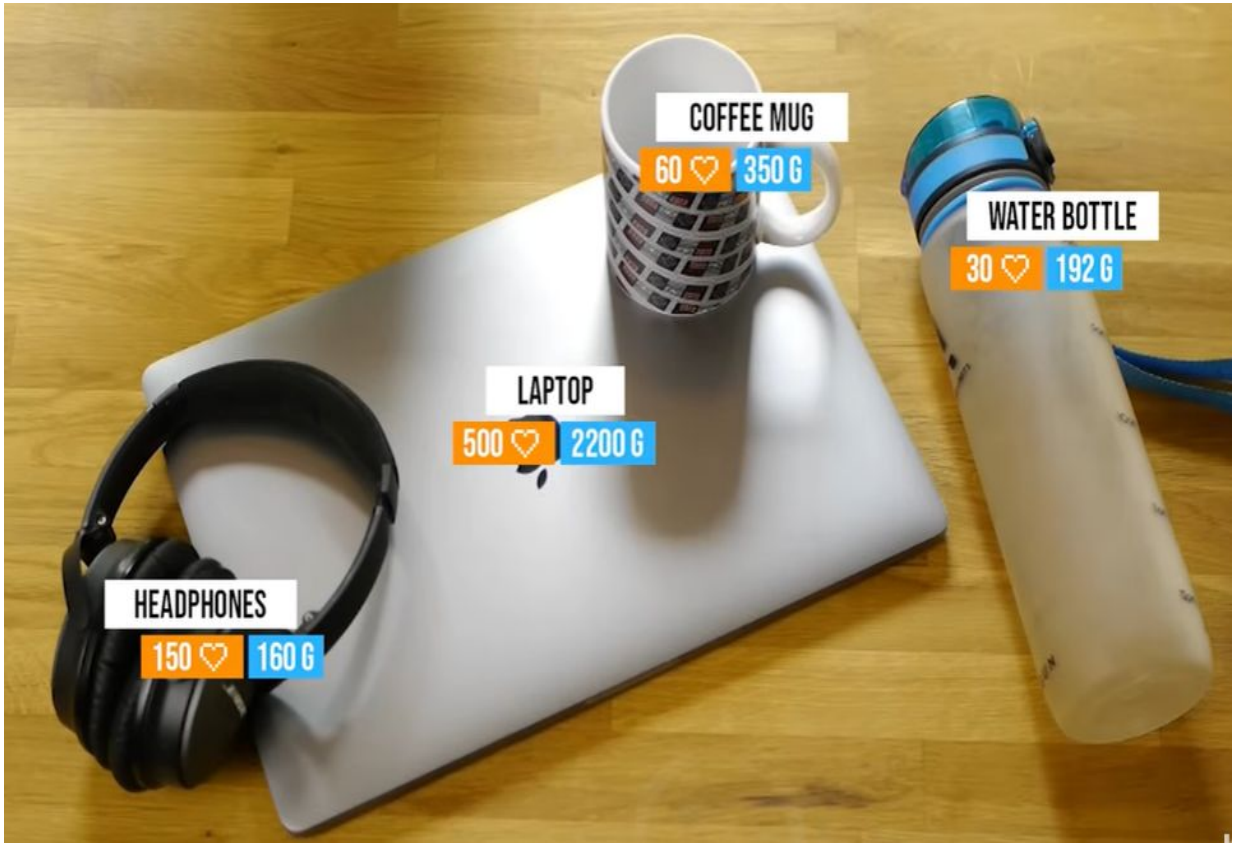
	Q-learning	Deep Q-learning
Approach	Tabular learning using Q-table	Function approximation with neural networks
Input	(state, action) pairs	Raw State input
Output	Q-values for each (state, action) pair	Q-values for each (state, action) pair
Training data	Q-table entries	Experience Replay buffer
Training time	Fast	Slow
Complexity	Limited by the number of states and actions	More complex due to the use of neural networks
Generalization	Limited to states in Q-table	Can generalize to unseen states
Scalability	Struggles with large state and action spaces	Handles large spaces well
Stability	Prone to overfitting	More stable than Q-learning, but can

Can you tell??



Combination of items with maximum utility subject to total weight = 3 kg?

Item count=5

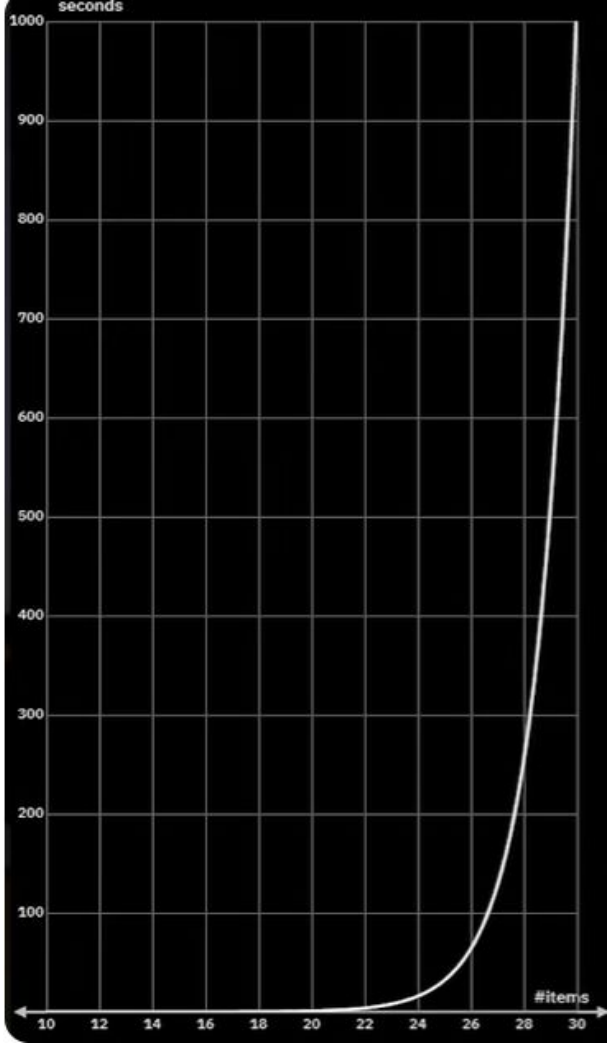

COFFEE MUG
60 ♡ 350 G
WATER BOTTLE
30 ♡ 192 G
LAPTOP
500 ♡ 2200 G
HEADPHONES
150 ♡ 160 G

Ans

Item count=10

BASEBALL CAP
100
70 G
COFFEE MUG
60
350 G
LAPTOP
500
2200 G
HEADPHONES
150
160 G
PHONE
500
200 G

Items	Combination	Time(in Seconds)
5	32	0.000003
10	1,024	0.000807
20	10,48,576	1.000000
21	20,97,152	1.940000



77 items will take 5 billion years!!!

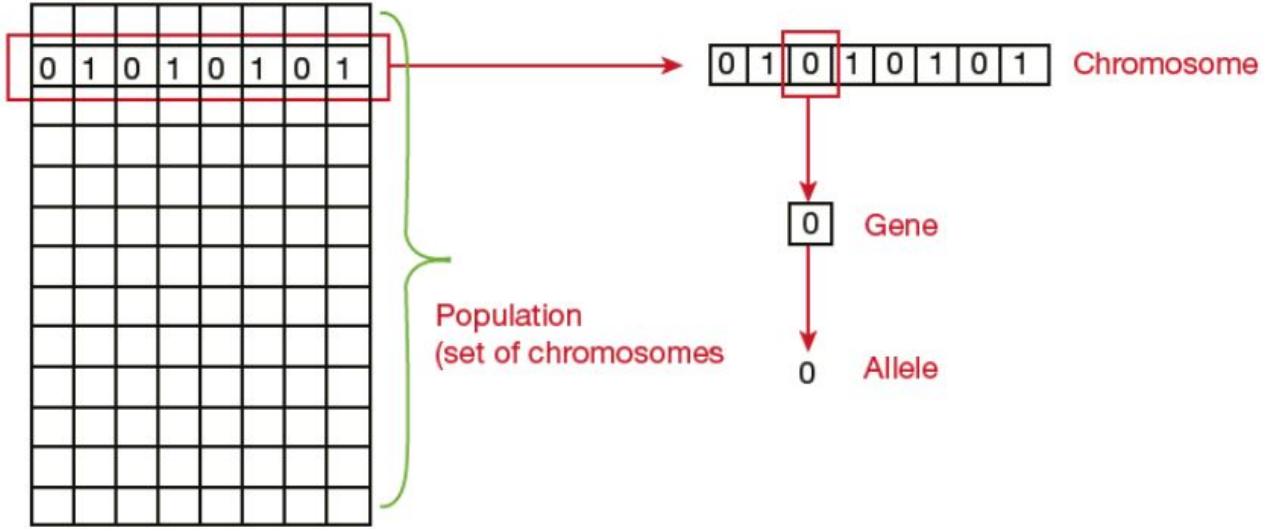
GENETIC ALGORITHMS: Introduction

- A **genetic algorithm** is a search-based optimization technique that draws its concept from the principles of **genetics and natural selection**.
- The main takeaway of these algorithms is that they can find optimal or near-optimal solutions to difficult optimization problems in research, and in machine learning problems. Genetic algorithms can find solutions to those problems which otherwise would take a lifetime to solve.
- This algorithm reflects the process of **natural selection** where the **fittest individuals are selected for reproduction** in order to produce offspring of the next generation
- Genetic Algorithms are being widely used in different real-world applications, for example, image processing, designing electronic circuits, code-breaking, and artificial creativity.

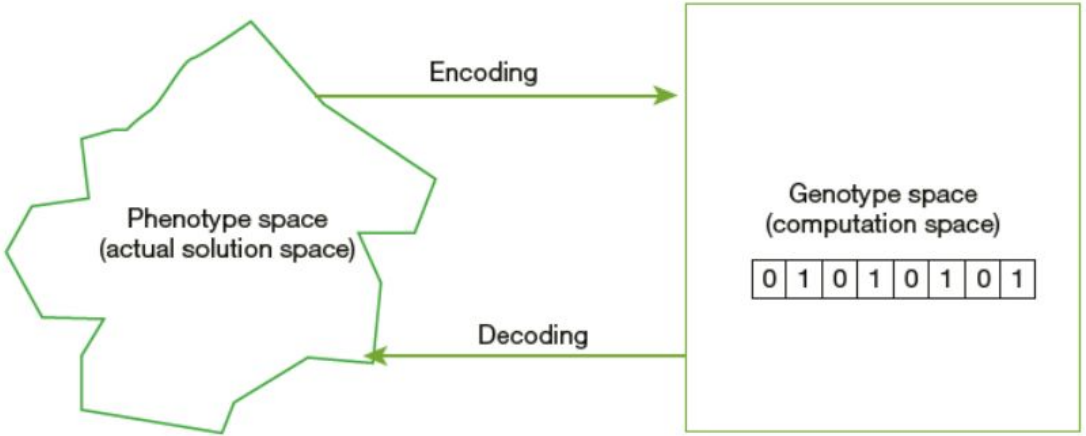
- We know that the search space comprises all possible solutions or values which the inputs can take. **The problem of optimization is to find those solutions or values in the search space that gives the optimal solution**.
- In genetic algorithms, the given problem has a **pool or a population of possible solutions**. Crossover and mutation are applied on these solutions to produce new children. This process is repeated over several generations. Centred around the Darwinian Theory of 'Survival of the Fittest', each individual (or candidate solution) thus produced is assigned a fitness value. Individuals having a higher fitness value are given a priority to mate and yield more 'fitter' individuals. This helps in the process of evolution that gives better solutions over generations, till stopping criterion is met.
- Though genetic algorithms are randomized, they perform much better than random local search as they exploit historical information as well.

Basic Terminology

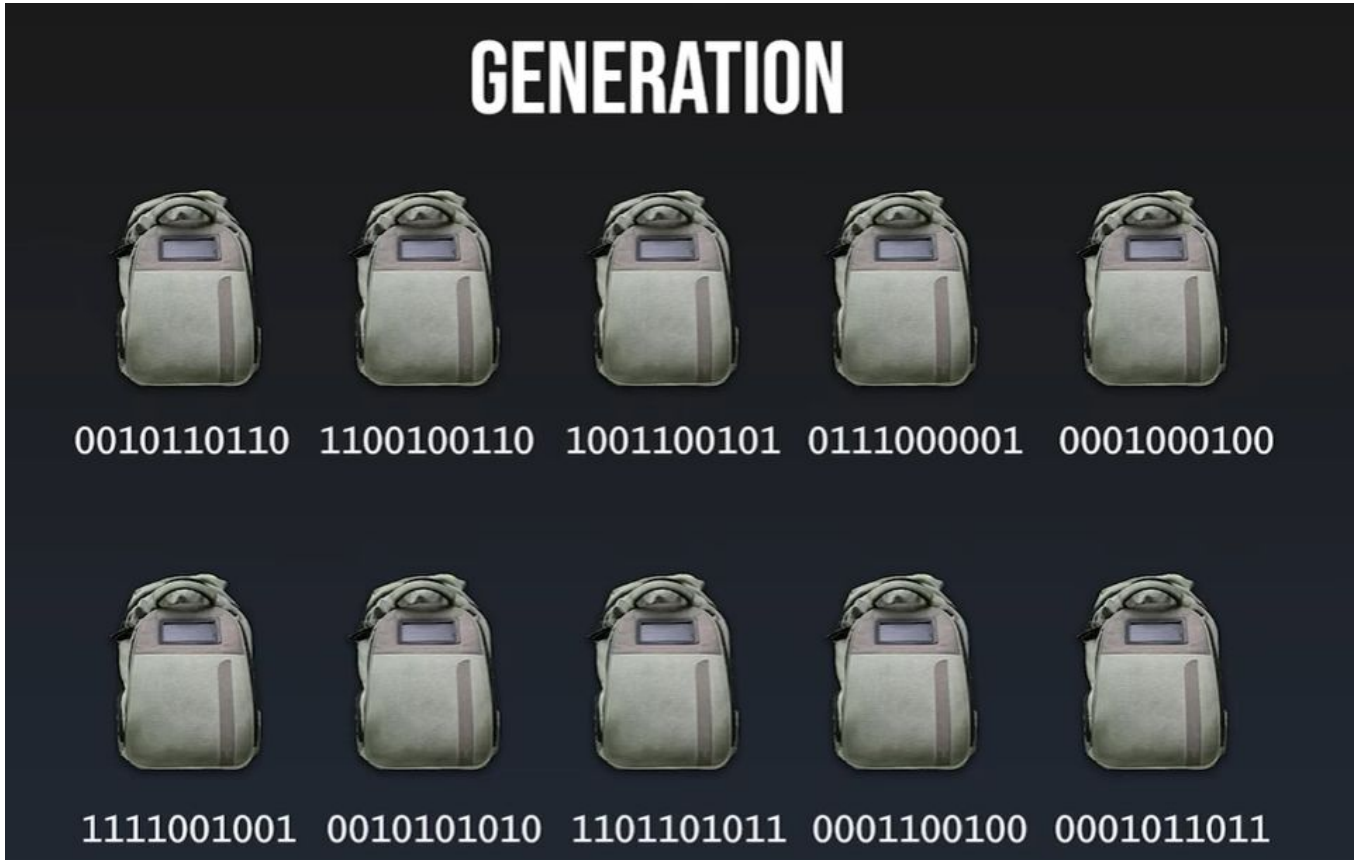
1. **Population:** It is a subset of all the possible solutions to the given problem.
2. **Chromosomes:** A chromosome is the individual solution to the given problem.
3. **Gene:** A gene is one element position of a chromosome.
4. **Allele:** It is the value which a gene takes for a particular chromosome.



5. **Genotype:** Genotype is the population (solution) in the computation space that can be represented in a simple and easy-to-understand manner so that it can be easily manipulated using a computing system.
6. **Phenotype:** Phenotype is the population (solution) in the real-world solution space that can be represented in real world situations.
7. **Decoding and Encoding:** For simple problems, the **phenotype and genotype** spaces are the same. But for complex, real-world problems, these spaces are different. While decoding is the process of transforming a solution from the genotype to phenotype space, encoding, on the other hand, transforms solution from the phenotype to genotype space. A genetic algorithm must perform decoding at a very fast pace as it has to be done repeatedly while calculating the fitness value.



FISHERMAN'S FRIEND
Tempo
1 0 0 1 1 0 0 1 0 1

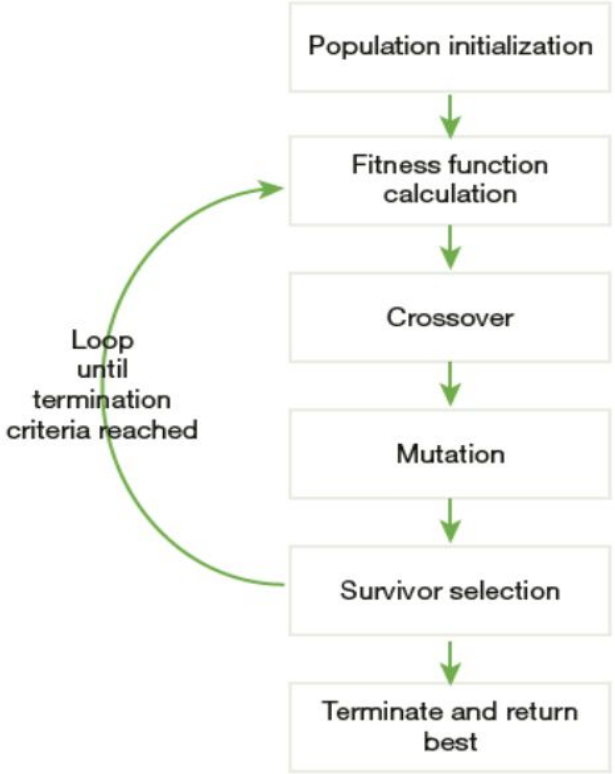

GENERATION
0010110110
1100100110
1001100101
0111000001
0001000100
1111001001
0010101010
1101101011
0001100100
0001011011

8. **Fitness Function** (or **Evaluation Function**): It is the function which accepts the solution as input and produces the suitability of the solution as the output. That is, the **fitness function** determines how fit a solution is. This is done by evaluating how close a given solution is to the optimum solution of the desired problem.

   In some cases, the fitness function and the objective function may be the same but usually they both are different. While the objective function is the function that has to be optimized, fitness function, on the other hand, is the function which guides the optimization.

9. **Genetic Operators:** Operators which change the genetic composition of the offspring are known as genetic operators. These include crossover, mutation, selection, etc.

Structure of Genetic Algorithm



GA Cycle of Reproduction

The typical **Genetic Algorithm cycle** includes:

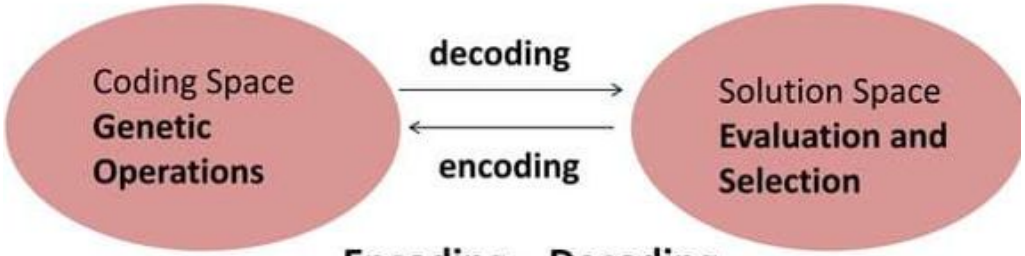
1. **Start:** Initialize a random population.
2. **Evaluate:** Calculate fitness for each individual.
3. **Select:** Choose parents based on fitness.
4. **Crossover:** Recombine parents to create offspring.
5. **Mutate:** Randomly alter offspring genes.
6. **Replace:** Form the new population with offspring (and sometimes the best of the old population).
7. **Repeat:** Continue until stopping criteria are met.

This cycle repeats until the algorithm converges to the best solution.

Encoding

Genetic algorithm works on two types of spaces alternatively, coding space (genotype) and solution space (phenotype). The phenotype describes the outward appearance of an individual.

A transformation exist between genotype and phenotype also called mapping.



Encoding – Decoding method

Permutation Encoding: Encoding schemes where fitness depends on order only.

Binary Encoding: Encoding schemes where fitness depends on value and order.

Value Encoding: Encoding schemes where fitness depends on value.

Binary Encoding: This is the most common form of encoding. In this each chromosome is represented using a binary string. In binary encoding every chromosome is a string of bits, 0 or 1.

Chromosome1	110101110010
Chromosome2	011010011101

✓ In this encoding each bit show some characteristic of solution. On the other side each binary string represents a value. With smaller number of alleles, a number of chromosomes can be represented.

✓ Crossover operations possible in binary encoding are 1-point crossover, N-point crossover, Uniform crossover and Arithmetic crossover. The Mutation operator possible is Flip. In Flip mutation, bits changes from 0 to 1 and 1 to 0 based on generated mutation chromosome.

<u>Octal Encoding</u>: In this encoding, each string uses a set of octal numbers (0 to 7).

- Example:

  Chromosome 1: 1534620

  Chromosome 2: 0772014

<u>Hexadecimal Encoding</u>: This encoding uses strings made up of hexadecimal numbers (0-9, A-F).

- Example:

  Chromosome 1: ZAD4F

  Chromosome 2: A05CE

<u>Permutation Encoding</u>: This encoding is used when solution involves an ordering or sequence of elements — where the position of elements matters and no element should repeat. This type is used in solving Traveling Salesman Problem (TSP), Job Scheduling, Ordering tasks or resources

Example:
Suppose we have 5 cities (A, B, C, D, E).

A possible chromosome could be [3, 5, 2, 1, 4]

<u>Value Encoding</u>: This encoding is used when the solution's variables can take real numbers, characters, or symbols, and it's not limited to binary or permutation representations.
This type is used in Function optimization, Neural network weight tuning
Each gene directly represents a value in the solution (often a real number or symbol).
Example:
For optimizing a function f(x,y,z) a chromosome could be: [2.5, -1.0, 3.8]
This means:x=2.5 y=−1.0 z=3.8

Fitness Function

The fitness function takes a candidate solution to the problem as input and produces a results statement as output. This output describes how 'fit' (or how 'good') the solution is with respect to the given problem. The fitness function should be fast as it will be repeatedly applied to find the fitness value. Hence, a slow fitness function can adversely affect the performance of a genetic algorithm. A good fitness function must possess the following characteristics.

1. It should be fast to compute.
2. It must quantitatively measure how fit a given solution is.

For some complex problems, it may not be possible to directly apply the fitness function to calculate the fitness value. In such cases, fitness approximation is done.

Genetic Operators

Genetic operators are used in genetic algorithms to guide the algorithm towards a solution for a given problem. These operators alter (change) the gene composition of the offspring (children). They work in conjunction with one another for the algorithm to be successful.

Types of Genetic Operators:

- Selection
- Crossover
- Mutation

Selection

- The selection step is one of the most important stages in a Genetic Algorithm (GA) — it decides which individuals (solutions) from the current population will be chosen to reproduce and create the next generation.
- The main goal is to favor better solutions (those with higher fitness) so that their genetic material is more likely to be passed on to future generations
- Each chromosome in the population is evaluated using a fitness function that measures how good the solution is.
- Chromosomes are chosen as parents based on their fitness — higher fitness → higher selection probability.
- The selected parents form a mating pool, from which crossover and mutation will produce offspring for the next generation.

Common Selection Methods

Different Techniques for Parent Selection:

- Fitness Proportionate Selection/ **Roulette Wheel Selection**
- Tournament Selection
- Rank Selection
- Random Selection

1. Roulette Wheel Selection (Fitness Proportionate Selection)

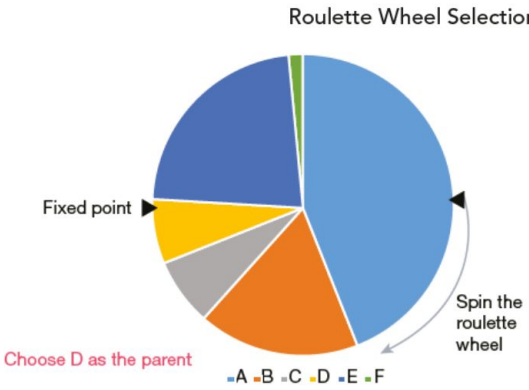
- Each individual gets a slice of a "roulette wheel" proportional to its fitness.
- The wheel is spun; individuals with higher fitness have a higher chance to be chosen.

Example:

If total fitness = 100

- A's fitness = 8.2 → 82% chance
- B's fitness = 3.2→ 32% chance
- C's fitness = 1.4 → 14% chance and so on

So, A is more likely to be selected

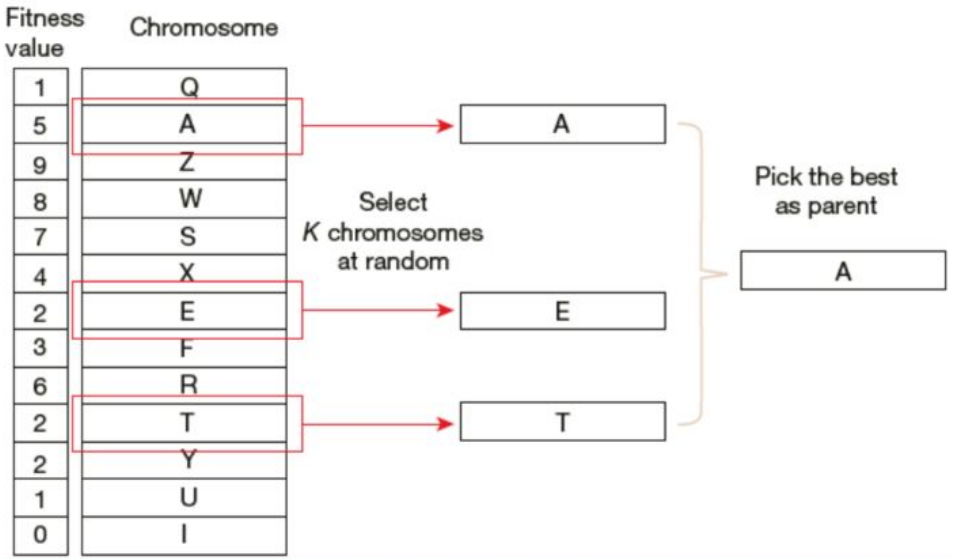


Chromosome	Fitness value
A	8.2
B	3.2
C	1.4
D	1.2
E	4.2
F	0.3

2. Tournament Selection

- Randomly pick a small group (say K individuals).
- The one with the **highest fitness** in that group is selected as a parent.
- Repeat the process until enough parents are chosen.

Advantages: Simple and works well even if fitness values are close.



3. **Rank Selection**

This is used when individuals in population have very close fitness value.

In this technique we rank the individuals in the population based on their fitness values. Therefore the selection of parents depends on the rank of each individual and not its fitness score.

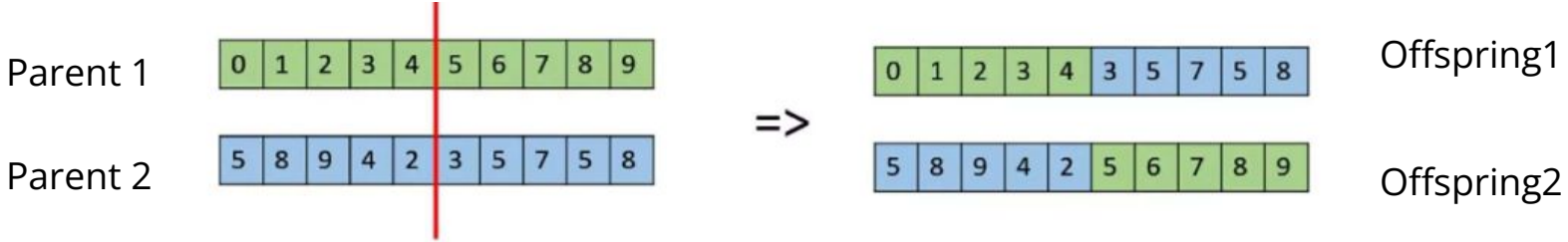
Higher the rank, higher is the probability of being chosen as a parent. Rank selection can work with negative fitness values as well.

Crossover

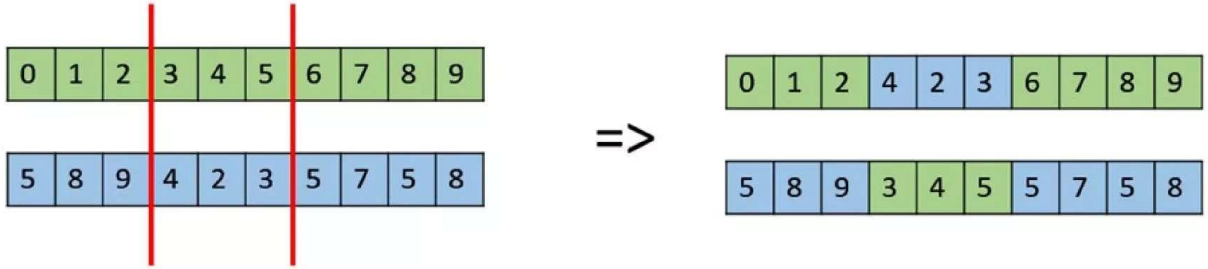
Crossover mimics biological reproduction where genes from two parents combine. In this technique, more than one parent is selected and one or more offsprings are produced using genetic material of parents.

Common Crossover Methods:

1. **Single-Point Crossover:**
   A random crossover point is chosen, and genes are swapped beyond that point.
   Example:



- Multi Point Crossover



- Uniform Crossover

Mutation

- Mutation is the process in which a new solution is obtained by introducing a small random tweak in the chromosome.
- Mutation plays an important role in genetic algorithms as it maintains as well introduces diversity in the genetic population.
- Mutations performs exploration of search space and is very essential for a genetic algorithm to converge successfully.

Example:

- Before Mutation: ABCDEHAF
- After Mutation: ABCDFHAF

GA- MUTATION

- used to maintain and introduce diversity in the genetic population
- Mutation Operators:

-Bit Flip Mutation

0	0	1	1	0	1	0	0	1	0

=>

0	0	1	0	0	1	0	0	1	0

-Random Resetting

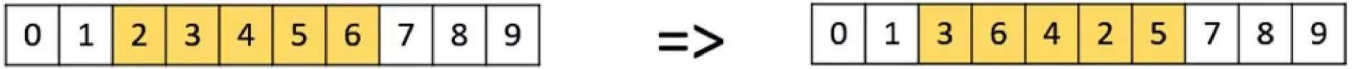
-Swap Mutation

1	2	3	4	5	6	7	8	9	0

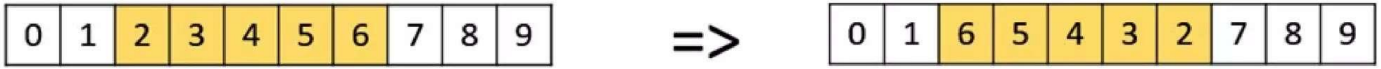
=>

1	6	3	4	5	2	7	8	9	0

- Scramble Mutation



- Inversion Mutation



Termination Condition

The **termination condition** of a Genetic Algorithm (GA) determines when the GA run should stop.

In the initial stages, GA improves solutions rapidly, but later improvements become smaller and slower (saturation phase).

The goal is to stop the algorithm when the solution is **close to optimal**.

Common termination conditions include:

1. **No improvement condition** – When there has been no improvement in the population for a specified number of iterations (X).

2. **Fixed generations condition** – When a predefined number of generations have been completed.

3. **Target fitness condition** – When the objective function reaches a specified threshold value.

A **counter** is used to track the number of generations without improvement:

- Initially, the counter is set to zero.
- If no offspring is better than the existing population, the counter is incremented.
- If any offspring has better fitness, the counter is reset to zero.
- The algorithm terminates when the counter reaches a **preset limit**.

The **choice of termination condition** is **problem-specific** and should be tuned experimentally by the GA designer for best results.

Genetic Programming (GP)

Genetic Programming is an extension of GAs where the chromosomes represent **computer programs** instead of numeric or binary strings.
It evolves programs that perform a specific task — for example, symbolic regression, decision trees, or control strategies.

Features:

- Each individual is a **program tree** (nodes = functions/operators, leaves = inputs/constants).
- Fitness is based on how well the program solves a problem.
- Crossover and mutation operate on subtrees instead of bits.

Example Application:
Evolving mathematical expressions that best fit a set of data points.